

Rate Limiting & Race Conditions — Interview Prep Notes

1. Logical Description

Rate Limiting (missing/weak)

- No/insufficient throttling on requests per user/IP/token/time window
- Allows unlimited or excessive repeated calls to sensitive endpoints

Race Conditions

- Time-of-Check-to-Time-of-Use (TOCTOU) flaw
 - Two+ concurrent requests exploit non-atomic operations (check→act gap)
 - Outcome depends on request timing/ordering, not business logic
-

2. OWASP Category & CWE

Vuln	OWASP (2021/API)	CWE
Rate Limiting	API4:2023 Unrestricted Resource Consumption; A04:2021 Insecure Design	CWE-770 (Allocation w/o Limits), CWE-799 (Improper Control of Interaction Frequency)
Race Condition	A04:2021 Insecure Design; API8:2023 (Security Misconfig, indirectly)	CWE-362 (Race Condition), CWE-367 (TOCTOU), CWE-841 (Improper Enforcement of Behavioral Workflow)

3. Common Variants

Rate Limiting

- Login/brute-force endpoints unthrottled
- OTP/password-reset unlimited attempts
- API resource exhaustion (no per-key/IP cap)
- GraphQL query cost unthrottled (nested queries)
- Global limit bypass via IP rotation/header spoofing (X-Forwarded-For)

Race Conditions

- Single-endpoint race (self-race) — e.g., redeem coupon N times
- Multi-endpoint race — e.g., balance check + withdraw
- Limit-overflow race — coupon/gift card, voting, inventory
- TOCTOU in file upload/delete

- Bypass MFA/step-up via parallel session state flip
-

4. Attack Surface / Entry Points

Rate Limiting

- Auth endpoints (login, OTP, password reset, MFA)
- Account creation/signup
- Search/export/report generation APIs
- Payment/transaction APIs
- Public APIs with API keys

Race Conditions

- Wallet/balance/coupon redemption
 - Inventory/stock decrement
 - Voting/like/follow counters
 - Account linking, referral bonus claim
 - Any multi-step transaction (check-then-update pattern)
-

5. Testing Workflow

Rate Limiting

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/Test
Baseline behavior	Is throttling present at all	Send N rapid requests via curl/loop	Repeater loop / Intruder (Sniper, no payload, just resend)	Fuzzer with delay=0	Yes — existing rate-limit-bypass templates	50-100 reqs in <10s
Threshold discovery	Find limit count/window	Binary search request count	Intruder w/ pitchfork (counter payload)	Fuzzer + response code tracking	N/A	Watch for 429/403 onset
Bypass via headers	IP/session spoof bypass	Add X-Forwarded-For, X-Real-IP, X-Client-IP per req	Intruder payload position on header value	Fuzzer header fuzzing	Custom template	Rotate IP-like values
Bypass via identifiers	Per-account vs per-IP enforcement	Rotate session tokens/cookies	Sequencer/Intruder with token list	Fuzzer w/ auth rotation	N/A	Multiple valid tokens
Distributed bypass	CDN/WAF-level gap	Use multiple source IPs (proxies)	Turbo Intruder for concurrency	N/A (limited concurrency)	N/A	High-concurrency burst

Race Conditions

Step	Verify	Manual	Burp	ZAP	Nuclei	Payload/Test
Identify candidate endpoint	Check-then-act logic exists	Review flow: balance check → deduct	N/A (analysis)	N/A	N/A	Coupon redeem, withdraw, transfer
Single-packet attack	True concurrency (network jitter removed)	N/A	Turbo Intruder (single-packet attack script) / Repeater "Send group in parallel"	N/A (no native single-packet)	N/A	20-50 identical requests
Multi-endpoint race	Race across 2 different endpoints	Scripted parallel curl/threads	Turbo Intruder multi-request	N/A	N/A	Balance-check + spend simultaneously
Session/state race	Token/session reuse timing	Parallel requests same token	Repeater group send	N/A	N/A	Password reset token used twice
Confirm impact	Did limit get bypassed (over-redeem, negative balance)	Check final state (DB/UI)	Response diffing	Response diffing	N/A	Compare expected vs actual count/balance

Note: Burp's **Turbo Intruder** (single-packet attack) is the gold standard — eliminates network jitter, near-simultaneous arrival at server.

6. Evidence to Collect

Rate Limiting

- Request/response logs showing N successful attempts, no 429/lockout
- Timestamps proving burst within short window
- Screenshot/video of automated tool output (count vs time)

Race Conditions

- Before/after state (DB values, balance, coupon usage count)
 - Burp Turbo Intruder response log — timestamps, response codes, response bodies showing duplicate success (200 OK) for same resource
 - Server response timing diffs
-

7. PoC Structure

Rate Limiting

- Script/Intruder attack sending X requests in Y seconds
- Show 100% success rate, no throttling response
- Demonstrate impact (e.g., brute-forced OTP, account takeover)

Race Conditions

- Turbo Intruder script with `engine=Engine.THREADED` or single-packet
 - Show N parallel redemption requests all returning 200
 - Show final state: coupon used N times instead of 1 / balance negative / stock oversold
-

8. Attack Chains

Rate Limiting

- No rate limit on OTP → brute-force OTP → account takeover
- No rate limit on login → credential stuffing → ATO
- No rate limit on password reset → token brute-force
- Resource exhaustion → DoS

Race Conditions

- Race on coupon redemption → financial loss
 - Race on withdraw + balance check → negative balance / fund duplication
 - Race on referral claim → unlimited bonus farming
 - Race combined with IDOR → duplicate resource creation across accounts
-

9. False Positives

Rate Limiting

- CDN/WAF rate limiting present but app-layer testing missed header (looks unlimited but isn't)
- Load balancer distributing to multiple app instances masking global limit
- Client-side throttling (JS) mistaken for server-side control

Race Conditions

- Network latency masking true concurrency (false negative more common — appears "not exploitable" when it actually is, due to poor tooling)
 - DB-level locking present but app logic re-checked before commit (false trigger)
 - Idempotency key present but not enforced correctly — needs deeper verification
-

10. Prerequisites for Exploitability

Rate Limiting

- No per-user/IP/session throttle at app or gateway layer
- No CAPTCHA/step-up after N failures
- Predictable/brute-forceable target (OTP, token)

Race Conditions

- Non-atomic check-then-act operation (read, decide, write as separate steps)
- No DB-level locking (no `SELECT FOR UPDATE`, no unique constraint, no atomic increment/decrement)
- No idempotency key / mutex / semaphore at app layer
- Ability to send concurrent requests (no WAF blocking parallel bursts)

11. Attacker Achievements

Rate Limiting

- Brute-force credentials/OTP/tokens → ATO
- Resource exhaustion → DoS
- Scraping/data harvesting at scale
- Spam account creation

Race Conditions

- Duplicate financial transactions (double-spend)
- Over-redemption of coupons/gift cards
- Inventory oversell / stock manipulation
- Bypass one-time-use restrictions (referral, trial)
- Privilege escalation via state flip race

12. CIA Impact

Vuln/Variant	Confidentiality	Integrity	Availability	Example Vector Fragment
Rate Limit — brute force login	High	Low	None	AC:L/PR:N/UI:N
Rate Limit — resource exhaustion	None	None	High	AC:L/PR:N/S:U/A:H
Rate Limit — OTP brute force	High	Medium	None	AC:L/PR:N/UI:N/C:H
Race — financial double-spend	None	High	None	AC:H/PR:L/UI:N/I:H
Race — inventory oversell	Low	High	Low	AC:H/PR:L/I:H
Race — privilege escalation	High	High	None	AC:H/PR:L/I:H/C:H

13. CVSS Severity & Vector

Vuln	Typical Score	Vector (v3.1)
Missing rate limit (login/OTP)	7.5–9.8 (High/Critical)	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Missing rate limit (DoS)	5.3–7.5	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
Race condition (financial)	7.5–9.1 (High/Critical)	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:H/A:N
Race condition (privilege)	8.1–9.1	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:N

- AC:H common for races (timing precision needed) — often debated, some raise to AC:L with tools like Turbo Intruder trivializing exploitation

14. Business Impact (Financial App Context)

Rate Limiting

- ATO at scale → fraud, regulatory reporting (PCI-DSS 8.1.6 - lockout after failed attempts)
- Credential stuffing → mass account compromise
- DoS on payment gateway → transaction downtime → SLA/revenue loss

Race Conditions

- Double-spend/duplicate transfer → direct monetary loss
- Coupon/gift-card abuse → financial leakage, audit findings
- Regulatory exposure: PCI-DSS (transaction integrity), SOX (financial controls), RBI/central bank guidelines (in India context) on transaction atomicity
- Reputational damage if exploited publicly (bug bounty disclosure)

15. Indicators/Signals

Rate Limiting

- No 429 Too Many Requests / no lockout after rapid repeated calls
- Missing `Retry-After`, `X-RateLimit-*` headers
- Same response time/behavior regardless of request volume

Race Conditions

- Check-then-act pattern visible in flow (balance read → separate write call)
- No idempotency-key requirement on state-changing endpoints
- Business logic allows "redeem"/"claim"/"transfer" actions repeatable
- DB schema lacks unique constraints/optimistic locking (version column)

16. Root Cause (Code Level)

Rate Limiting

```
// No counter/throttle check before processing
handleLogin(req):
    verifyCredentials(req.user, req.pass)    // no attempt counter, no delay, no logout
    return response
```

- Missing middleware/gateway-level throttle
- Rate limit logic only client-side (JS) or absent entirely
- Per-endpoint limits not defined in API gateway config

Race Conditions

```
// Classic TOCTOU
balance = getBalance(userId)           // CHECK
if balance >= amount:
    updateBalance(userId, balance - amount)    // ACT - non-atomic, gap between check & wr
ite
```

- Read and write are separate DB round trips
- No `SELECT ... FOR UPDATE`, no transaction isolation (READ COMMITTED without locking)
- No unique constraint preventing duplicate `couponid` + `userid` row

17. Remediation & Secure Design

Rate Limiting

- Enforce server-side rate limiting (per user, IP, API key) at gateway/WAF/app layer
- Sliding window/token bucket algorithm
- Progressive delays + account logout + CAPTCHA after N attempts
- Return `429` with `Retry-After` header

Race Conditions

- Atomic operations: DB-level `SELECT FOR UPDATE`, atomic increment/decrement (`UPDATE ... SET balance = balance - X WHERE balance >= X`)
- Optimistic locking (version column, compare-and-swap)
- Idempotency keys for state-changing requests
- Application-level mutex/distributed lock (Redis Redlock) for cross-service races
- Unique DB constraints to prevent duplicate claims (e.g., `unique(userid, couponid)`)

- Serializable transaction isolation where correctness > throughput
-

18. Compensating Controls (if root cause unfixed)

Rate Limiting

- WAF/CDN-level rate limiting (Cloudflare, AWS WAF) as stopgap
- Bot detection/CAPTCHA
- Alerting on abnormal request volume

Race Conditions

- Post-transaction reconciliation jobs (detect anomalies, auto-flag/reverse)
 - Manual review threshold triggers on high-value transactions
 - Rate-limit the specific endpoint to reduce race window exploitability (indirect mitigation)
-

19. Recurrence — Security Posture Signal

- Indicates **insecure design phase gap** — no threat modeling for concurrency/abuse cases (AO4:2021 root theme)
 - SDLC lacks concurrency/load testing in QA
 - No secure design checklist enforcing atomic transactions for financial/state-changing logic
 - Absence of API gateway standard limiting configuration across services → inconsistent enforcement
-

20. Upstream/SAST/Design Improvements

Rate Limiting

- Framework-level middleware enforced by default (e.g., express-rate-limit, Spring RateLimiter) as shared library
- API Gateway policy templates (Kong, Apigee) mandatory per route
- Design checklist: "Does this endpoint require rate limiting?" at design review

Race Conditions

- Semgrep/CodeQL rules: flag read-then-write DB pattern without transaction/lock
 - Linter rule: flag balance/counter updates not using atomic SQL operations
 - Design checklist: "Is this operation idempotent? Does it need locking?"
 - Mandatory code review gate for any financial/state-mutating endpoint
-

21. Sample Triage Response

Rate Limiting

"Confirmed — the `/api/v1/otp/verify` endpoint accepts unlimited verification attempts without lockout or delay, enabling OTP brute-force. Reproduced with N requests in $<30s$, all processed without 429. Rated High (CVSS 8.1) per API4:2023. Escalating to eng for gateway-level throttling."

Race Conditions

"Confirmed — concurrent requests to `/api/v1/coupon/redeem` using Turbo Intruder single-packet attack allowed the same coupon to be redeemed 15x instead of once, due to non-atomic balance check. Direct financial impact. Rated Critical (CVSS 9.1). Recommend atomic DB transaction + unique constraint as fix."

22. Sample Developer Remediation Note

Rate Limiting

"Add server-side rate limiting middleware to `/api/v1/otp/verify` — max 5 attempts per user per 15 min, lock account + require CAPTCHA on 6th attempt. Return 429 with Retry-After header. Do not rely on client-side throttling."

Race Conditions

"Refactor `redeemCoupon()` to use a single atomic SQL statement (`UPDATE coupons SET used=true WHERE id=? AND used=false`) instead of separate read-check-write. Add unique constraint on (couponid, userid). Wrap in DB transaction with row-level lock (`SELECT FOR UPDATE`) if multi-step logic is unavoidable."

23. Real-World CVE / Bug Bounty Examples

Rate Limiting

- Instagram password-reset brute-force (no rate limit on 6-digit code) — public bug bounty writeup, allowed brute-forcing within ~10 min
- GitHub — historic reports on missing rate limits on 2FA recovery codes

Race Conditions

- Starbucks gift card race condition (bug bounty) — race condition allowed transferring balance between cards to generate free funds
- Shopify — reported race condition in discount code redemption allowing multiple uses
- HackerOne general class — "TOCTOU in payment/wallet APIs" recurring theme across fintech programs

- CVE-2021-3156 (Sudo Baron Samedit) — race/buffer overflow class, cited generally in race-condition discussions though heap-overflow primary root cause
-

30-second interview answer: *"Rate limiting failures let attackers brute-force or exhaust resources due to missing per-user/IP throttling. Race conditions exploit the gap between a check and an act — like balance-check-then-deduct — where concurrent requests bypass business logic because operations aren't atomic. I test both with Burp's Turbo Intruder for true concurrency, verify via before/after state diffing, and remediation is server-side rate limiting plus atomic DB operations (locking, unique constraints, idempotency keys)."*